

Análisis de Sistemas

Materia:
Análisis y Metodología de
Sistemas

Docente contenidista: QUIRÓZ, Gamaliel

Revisión: Coordinación

Contenido

Introducción	04
Microservicios	05
¿Qué son los microservicios?	05
Arquitectura monolítica vs. microservicios	07
Consideraciones	07
¿Cómo se construye un microservicio?	08
Consideraciones técnicas	08
Beneficios de los microservicios	09
Desafíos de los microservicios	09
En resumen	10
Librerías que utilizaremos para el auth	11
Manejo de sesiones con Flask-Login	12
Manejo manual de session en Flask	14
Conexión de todo el flujo (registro + login + perfil)	15
Probado con Postman:	17
Bibliografía	20

Clase 9



¡Te damos la bienvenida a la materia
Análisis y Metodología de Sistemas!

En esta clase vamos a ver los siguientes temas:

- Definición a los microservicios.
- Arquitectura monolítica vs. los microservicios.
- Construcción de un microservicio.
- Autenticación y hashéo de contraseñas con Flask.

Introducción

En la evolución del desarrollo de software, uno de los principales desafíos ha sido crear aplicaciones escalables, mantenibles y seguras. Durante años, la arquitectura monolítica fue el enfoque predominante, en donde todas las funcionalidades de un sistema se encuentran integradas en un único bloque.

Con el aumento en la complejidad de las aplicaciones modernas (por ejemplo, sistemas de comercio electrónico, redes sociales o plataformas de streaming), surge la necesidad de un enfoque más modular. Aquí es donde aparecen los microservicios: una forma de dividir una aplicación en servicios pequeños e independientes, que se comunican entre sí.

En esta clase vamos a:

- Entender qué son los microservicios.
- Analizar sus ventajas frente a la arquitectura monolítica.
- Ver ejemplos prácticos aplicados con Python y Flask.
- Explorar librerías que permiten la autenticación y manejo de sesiones.

Microservicios

¿Qué son los microservicios?

Un microservicio es una unidad pequeña e independiente dentro de una aplicación más grande. Cada microservicio tiene una única responsabilidad clara (por ejemplo, “gestión de usuarios” o “procesamiento de pagos”) y se comunica con otros servicios mediante APIs REST o sistemas de mensajería.

Ejemplo:

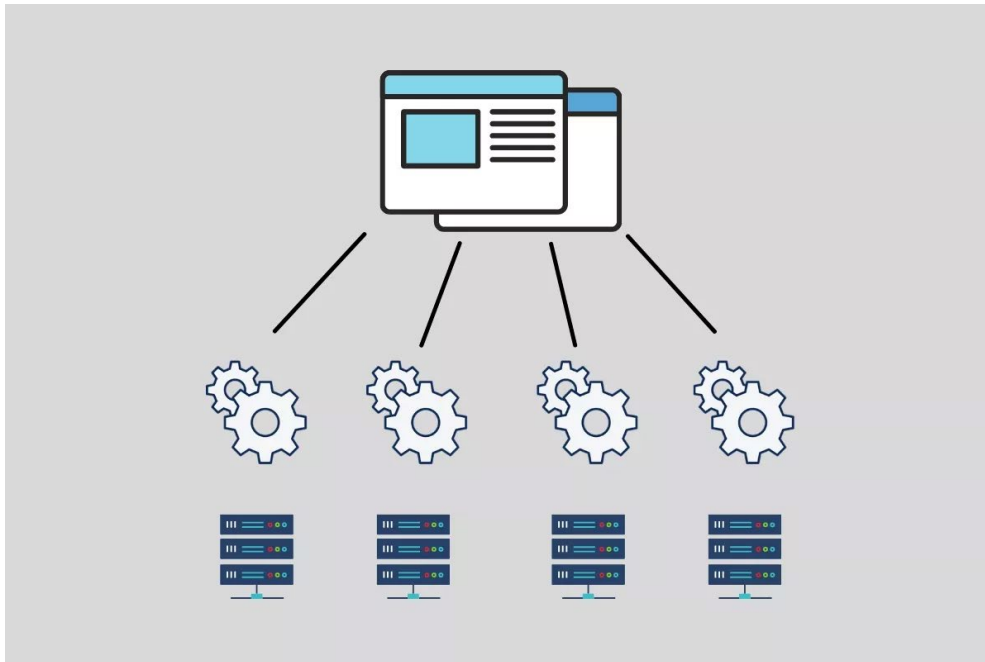
Imaginemos una pizzería online. En lugar de tener un solo sistema que haga todo, podemos dividirlo en:

- Un servicio para recibir pedidos.
- Un servicio para gestionar el stock de ingredientes.
- Un servicio de facturación.
- Un servicio de notificaciones para avisar al cliente que su pizza está lista.

Cada microservicio es como un empleado especializado de la pizzería: se enfoca en una tarea concreta.

Los **microservicios** son un enfoque de arquitectura de software que propone construir aplicaciones dividiéndolas en pequeños **servicios independientes**, que se comunican entre sí a través de **APIs** (normalmente HTTP/REST o mensajería).

En lugar de un único sistema grande (monolito), se crean **módulos autónomos**, cada uno con una única responsabilidad y que puede funcionar por separado.

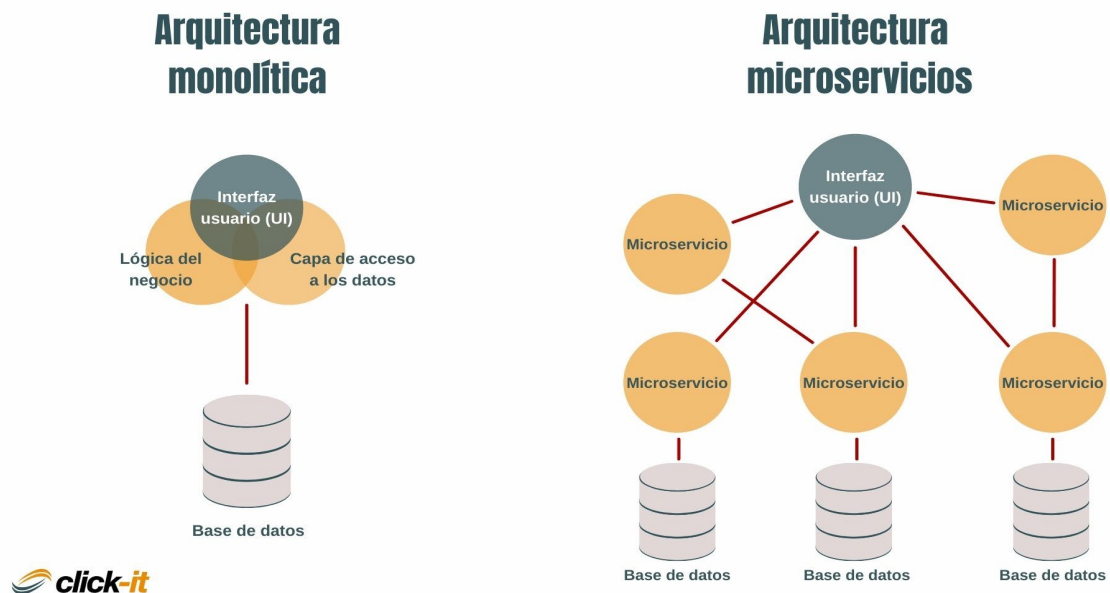


En lugar de crear una aplicación *monolítica* (donde todo está integrado en un solo bloque), los microservicios *dividen* las funcionalidades en servicios pequeños y autónomos. Esto significa que cada servicio puede desarrollarse, desplegarse y escalar de manera independiente.

Ventajas de los microservicios

- Autonomía: cada servicio se desarrolla y despliega por separado.
- Escalabilidad: se pueden escalar los módulos más usados sin necesidad de duplicar toda la aplicación.
- Flexibilidad tecnológica: cada microservicio puede estar escrito en un lenguaje distinto.
- Mantenimiento sencillo: si falla un servicio, el resto sigue funcionando.
- Reutilización: un servicio creado puede ser utilizado en múltiples aplicaciones.

Arquitectura monolítica vs. microservicios



Consideraciones

Realmente a la hora de crear un microservicios es **importante** poder determinar cómo crear cada uno de ellos y su relevancia.

El ejemplo que veremos se encargará de autenticar a los usuarios y devolver un token si las credenciales son correctas.

Ejemplo clásico

En una aplicación de comercio electrónico podríamos tener:

Un microservicio para **autenticación de usuarios**.

Otro para **productos y catálogo**.

Uno más para **carrito de compras**.

Otro separado para **pagos y facturación**.

Cada uno se puede construir y desplegar **independientemente**.

¿Cómo se construye un microservicio?

1. **Identificar una responsabilidad clara** (por ejemplo: "login", "productos", "clima").
2. Crear una **API REST** que exponga esa funcionalidad.
3. Usar una base de datos propia o compartida (según el caso).
4. Manejar la **comunicación** con otros servicios (ej: vía HTTP o mensajes).
5. Proteger con mecanismos como **JWT** o autenticación por token.

Consideraciones técnicas

- Cada servicio puede tener su **base de datos propia**.
- Si uno falla, el otro puede seguir funcionando (desacople).
- Necesitas definir bien cómo se comunican (API pública, contratos, seguridad).
- Podés usar contenedores (como Docker) para separarlos físicamente.

Diferencias monolíticas vs. Microservicios

Aspecto	Monolítica	Microservicios
Despliegue	Todo junto, un único bloque	Independiente por servicio
Escalabilidad	Escala todo el sistema	Escala solo lo necesario
Lenguajes/tecnologías	Uno solo	Diferentes por servicio
Fallos	Un error puede detener toda la app	Fallo aislado, el resto sigue funcionando
Ejemplo	App de e-commerce completa en un solo proyecto	Servicios separados: login, catálogo, pagos

Beneficios de los microservicios

- Escalabilidad independiente.
- Flexibilidad tecnológica (cada equipo puede usar distintos lenguajes o frameworks).
- Despliegue rápido y continuo.
- Mejor resiliencia ante fallos.

Desafíos de los microservicios

- Complejidad en la comunicación y coordinación.
- Necesidad de infraestructura robusta (monitoreo, logging, orquestación).
- Gestión de seguridad distribuida.
- Mayor carga en la configuración inicial y DevOps.

En resumen

Los microservicios nos permiten **dividir responsabilidades**, construir partes de la aplicación más robustas y seguras, y escalar según la necesidad de cada servicio.



En este ejemplo concreto, vamos a usar Flask para crear un microservicio de autenticación y otro de clima, mostrando cómo se conectan entre sí y responden sólo si el usuario está autenticado.

Librerías que utilizaremos para el auth

Encriptación de contraseñas con Flask-Bcrypt

Nunca debemos guardar contraseñas en texto plano.

En vez de eso, usamos bcrypt, un algoritmo diseñado para el hashing seguro de contraseñas.

¿Cómo lo instalamos? Ejecutando el comando:

pip install flask-bcrypt

Ejemplo:

```
• (.venv) PS D:\Python\Projects-In-Python\AyM_Python_DV\EncriptacionFlask_DV> pip install flask-bcrypt
Collecting flask-bcrypt
```

¿Cómo lo usamos en Python entonces? De la siguiente manera:

```
1  from flask_bcrypt import Bcrypt
2
3  bcrypt = Bcrypt()
4
5  # 1) Cuando un usuario se registra:
6  password = "mi_contraseña_segura123"
7  hashed_pw = bcrypt.generate_password_hash(password).decode('utf-8')
8
9  print("Contraseña en texto plano:", password)
10 print("Contraseña encriptada (hash):", hashed_pw)
```

- **generate_password_hash:** transforma la contraseña en un hash irreversible.
- **.decode('utf-8'):** lo convierte a string (ya que el hash viene en bytes).
- No se puede volver atrás -> ni el programador puede recuperar la contraseña original.

Validación en el Login:

```

12  # Contraseña guardada (hash) en la base de datos
13  stored_pw = hashed_pw
14
15  # Contraseña que escribe el usuario en login
16  input_pw = "mi_contraseña_segura123"
17
18  if bcrypt.check_password_hash(stored_pw, input_pw):
19      print("Login correcto")
20  else:
21      print("Contraseña incorrecta")

```

Explicación:

- **check_password_hash:** compara el hash guardado con lo que escribió el usuario.
- Si coinciden -> login válido.

Nunca guardamos contraseñas en texto plano.

Manejo de sesiones con Flask-Login

Nos ayuda a manejar quién está logueado, proteger rutas y trabajar con `current_user`.

La manera de poder utilizarlo en nuestro entorno es ejecutando el comando "pip install flask-login" de la siguiente manera:

```

(.venv) PS D:\Python\Projects-In-Python\AyM_Python_DV\EncriptacionFlask_DV> pip install flask-login
Collecting flask-login

```

Ejemplo sencillo de cómo se implementa:

```

1  from flask import Flask
2  from flask_login import LoginManager, UserMixin, login_user, logout_user, login_required, current_user
3
4  app = Flask(__name__)
5  app.secret_key = "supersecretkey"
6
7  # Configurar LoginManager
8  login_manager = LoginManager()
9  login_manager.init_app(app)

```

```

11 # Simulación de usuarios (trae de una BD)
12 class User(UserMixin):
13     def __init__(self, id, username):
14         self.id = id
15         self.username = username
16
17 # diccionario simulando la base de datos
18 users = {"elias": User(1, "elias")}
19
20 @login_manager.user_loader
21 def load_user(user_id):
22     # Flask-Login necesita cargar el usuario desde la BD por ID
23     for user in users.values():
24         if str(user.id) == str(user_id):
25             return user
26     return None
27
28 @app.route("/login")
29 def login():
30     user = users["elias"]
31     login_user(user) # Marca al usuario como logueado
32     return f"Usuario {user.username} ha iniciado sesión"
33

```

```

34 @app.route("/logout")
35 @login_required
36 def logout():
37     logout_user()
38     return "Sesión cerrada"
39
40 @app.route("/perfil")
41 @login_required
42 def perfil():
43     return f"Bienvenido {current_user.username} a tu perfil privado"

```

Explicación:

- login_user(user) → guarda al usuario en sesión.
- logout_user() → cierra sesión.
- @login_required → protege la ruta, obliga a estar logueado.
- current_user → accede al usuario actual.

Manejo manual de session en Flask

Flask, además de Flask-Login, permite guardar datos en cookies firmadas usando session.

```
1  from flask import Flask, session
2
3  app = Flask(__name__)
4  app.secret_key = "clave_muy_secreta"
5
6  @app.route("/set_session")
7  def set_session():
8      session["user_id"] = 123
9      return "Se guardó user_id en la sesión"
10
11 @app.route("/get_session")
12 def get_session():
13     user_id = session.get("user_id")
14     return f"El user_id en sesión es: {user_id}"
15
16 @app.route("/clear_session")
17 def clear_session():
18     session.clear()
19     return "Sesión borrada"
```

- **session["clave"] = valor** → guarda un valor en la cookie.
- La cookie se firma con **SECRET_KEY**, entonces el usuario no puede modificarla sin invalidarla.
- Diferencia con Django/PHP -> ellos guardan sesión en disco/BD, Flask lo hace en cookie.

Conexión de todo el flujo (registro + login + perfil)

Link de acceso al código completo:

https://docs.google.com/document/d/1Sz94P_FsLr9vNBwltRDpyq48sSE30c-yz0ZMfgIgU8/edit?usp=sharing

```
You, 2 days ago | 1 author (You)
1  from flask import Flask, request, redirect, url_for
2  from flask_bcrypt import Bcrypt
3  from flask_login import LoginManager, UserMixin, login_user, logout_user, login_required, current_user
4
5  app = Flask(__name__)
6  app.secret_key = "supersecretkey"
7
8  bcrypt = Bcrypt()
9  login_manager = LoginManager()
10 login_manager.init_app(app)
11
12 # Base de datos simulada
13 users = {}
14
You, 2 days ago | 1 author (You)
15 class User(UserMixin):
16     def __init__(self, id, username, password_hash):
17         self.id = id
18         self.username = username
19         self.password_hash = password_hash
20
```

```
21 @login_manager.user_loader
22 def load_user(user_id):
23     return users.get(int(user_id))
24
25 @app.route("/register", methods=["POST"])
26 def register():
27     username = request.form["username"]
28     password = request.form["password"]
29     hashed_pw = bcrypt.generate_password_hash(password).decode("utf-8")
30     user = User(len(users) + 1, username, hashed_pw)
31     users[user.id] = user
32     return "Usuario registrado"
33
34 @app.route("/login", methods=["POST"])
35 def login():
36     username = request.form["username"]
37     password = request.form["password"]
38
39     for user in users.values():
40         if user.username == username and bcrypt.check_password_hash(user.password_hash, password):
41             login_user(user)
42             return "Login exitoso"
43     return "Usuario o contraseña incorrectos"
```



```

44
45 @app.route("/perfil")
46 @login_required
47 def perfil():
48     return f"Bienvenido {current_user.username}"
49
50 @app.route("/logout")
51 @login_required
52 def logout():
53     logout_user()
54     return "Sesión cerrada"
55
56
57 if __name__ == "__main__":
58     app.run(debug=True)

```

Registro (/register)

- El usuario manda un username y una password.
- La contraseña no se guarda en texto plano, sino que se encripta con bcrypt y se almacena el hash.
- Esto significa que si alguien roba la base de datos, no ve las contraseñas reales.
- En este ejemplo usamos un diccionario en memoria (no base de datos real), pero la lógica es la misma.

Login (/login)

- El usuario envía sus credenciales (username y password).
- Flask busca ese usuario en la base (nuestro diccionario).
- Se compara la contraseña escrita con el hash almacenado.
- Si coincide, usamos login_user() de Flask-Login, que guarda la sesión del usuario en una cookie firmada.
- A partir de ese momento, Flask ya sabe "quién está logueado".

Perfil (/perfil)

- Es una ruta protegida con @login_required.
- Solo se puede acceder si el usuario ya inició sesión.
- Si está logueado, mostramos su nombre usando current_user.
- Si no está logueado, Flask redirige a login o devuelve error.

Logout (/logout)

- Cierra la sesión del usuario.
- Esto significa que la cookie de sesión queda inválida y ya no puede entrar a rutas protegidas hasta que vuelva a hacer login.

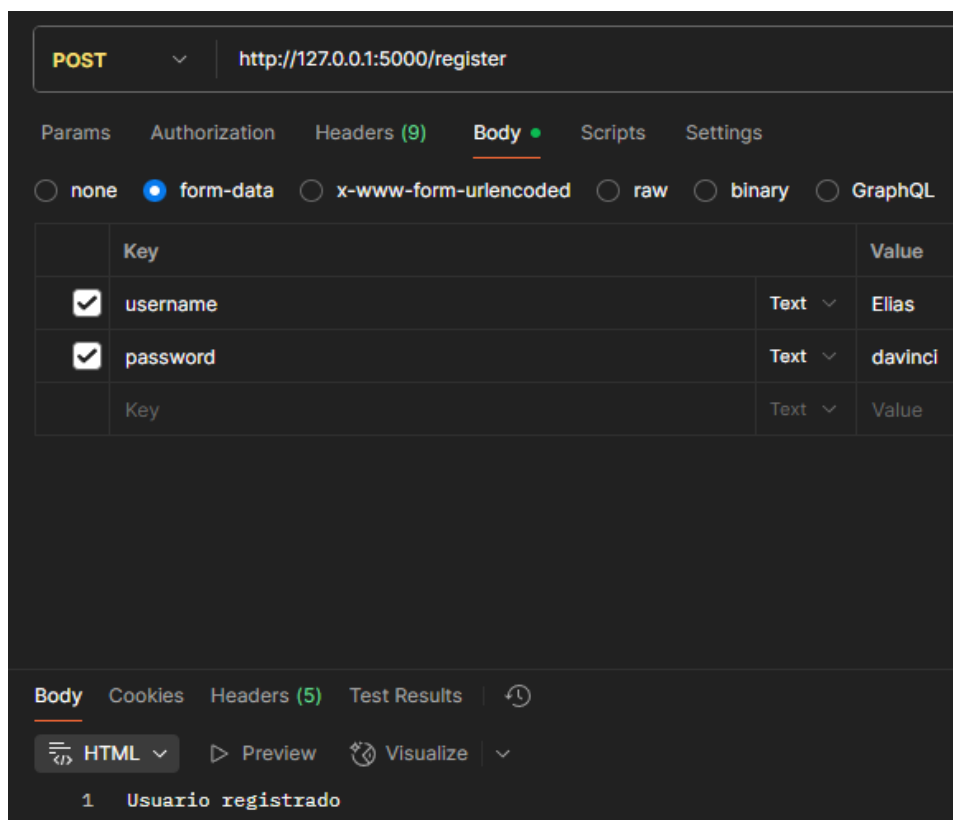
Registro: *"Le digo al sistema quién soy y qué contraseña quiero usar. El sistema la guarda en forma encriptada, nunca en texto plano."*

Login: *"Le digo al sistema mi usuario y contraseña, y el sistema chequea si la contraseña coincide con el hash guardado."*

Perfil: *"Como ya estoy autenticado, el sistema sabe quién soy y me deja entrar a mi zona privada."*

Logout: *"Cierro mi sesión, el sistema borra mi cookie, y ya no tengo acceso a zonas privadas hasta volver a loguearme."*

Probado con Postman:



Qué hacemos en Postman:

- Enviamos datos con POST en lugar de escribir la URL en el navegador.
- Los navegadores hacen GET por defecto.
- Como el registro y login necesitan datos (usuario + contraseña), usamos Postman para enviarlos como si fueran formularios.

Registrar usuario

- Enviamos un POST a /register con:
- username = elias
- password = 1234
- Flask recibe esto, encripta la contraseña y guarda al usuario.
- Respuesta: "Usuario registrado".

Login

- Enviamos un POST a /login con el mismo usuario y contraseña.
- Flask compara el password ingresado con el hash guardado.
- Si coincide → Respuesta: "Login exitoso".
- Además, Flask-Login guarda la sesión en una cookie para saber que este cliente está autenticado.

Entrar al perfil protegido

- Ahora hacemos un GET a /perfil.
- Como la cookie de sesión está activa, Flask sabe que el usuario está logueado.
- Respuesta: "Bienvenido Elias".

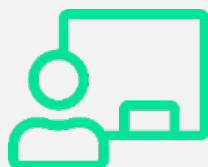
Logout

- Hacemos un GET a /logout.
- Flask elimina la sesión del usuario.
- Respuesta: "Sesión cerrada".
- Si intentamos entrar de nuevo a /perfil, ya no nos deja.



Hemos llegado así al final de esta clase en la que vimos:

- Definición a los microservicios.
- Arquitectura monolítica vs. los microservicios.
- Construcción de un microservicio.
- Autenticación y hash de contraseñas con Flask.



Te esperamos en la **clase en vivo** de esta semana.
No olvides realizar el **desafío semanal**.

¡Hasta la próxima clase!

Bibliografía

Fowler, M., & Lewis, J. (2014). *Microservices: A definition of this new architectural term*. MartinFowler.com.

<https://martinfowler.com/articles/microservices.html>

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

Grinberg, M. (2018). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media.

Rahman, M., & Rahman, M. (2021). *Microservices with Docker, Flask, and React*. Apress.

Villamizar, M., Garcés, O., Ochoa, L., Castro, H., Salamanca, L., Verano, M., & Lang, M. (2017). Cost comparison of running web applications in the cloud using monolithic, microservice, and AWS architectures. *IEEE Latin America Transactions*, 15(2), 399–406.

<https://doi.org/10.1109/TLA.2017.7854630>

Flask-HTTPAuth. (n.d.). *Flask-HTTPAuth documentation*.

<https://flask-httpauth.readthedocs.io/en/latest>